FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Guilherme Dias dos Santos

Produto: **Kreato**
https://github.com/guilherme-dds/kreato

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 5
3.4 Fluxo de assinatura 6
3.5 Modelo de dados do domínio 7
3.6 Segurança 8
3.7 Observabilidade, CI/CD e documentação da API 9
3.8 Camadas IaaS, PaaS e SaaS do projeto 10
4 Quadro de entregas 10
5 Atividades realizadas 11
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 11
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 12
5.3 Atividade 9 — Fork e plano de negócio 14
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 17
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 19
6 Considerações finais 21
7 Declaração de autoria 21

1 Introdução

A disciplina Introdução à Computação em Nuvem (ISW055) trabalha, de forma prática, a construção e a operação de uma aplicação distribuída em contêineres: começa com um sistema simples e, a cada atividade, acrescenta uma peça típica de um produto em nuvem — microsserviço de autenticação, controle de acesso por papel, auditoria com Redis, armazenamento de objetos com MinIO, cobrança com Stripe — sempre implantada na mesma stack do Portainer, numa VM e num domínio fornecidos pela Fatec.

No 1º bimestre esse caminho foi percorrido com um catálogo de filmes (Tom Hanks). No 2º bimestre a proposta mudou de escala: em vez de mais uma funcionalidade isolada, o catálogo foi transformado em um SaaS de verdade. Primeiro, o catálogo ganhou um plano pago com o Stripe (atividade 7) e uma documentação reversa com a classificação das camadas IaaS, PaaS e SaaS (atividade 8). Depois, o repositório foi copiado com todo o histórico para um repositório novo (atividade 9), o domínio de filmes foi trocado pelo do negócio (atividade 10) e, por fim, o produto recebeu planos recorrentes, limites por plano e assinatura completa (atividade 11).

O produto escolhido é o **Kreato**, uma central de gestão e produção de conteúdo para criadores — de iniciantes a pequenas equipes que publicam no TikTok, Instagram e YouTube. A dor que ele resolve é a fragmentação: ideias ficam no bloco de notas, roteiros em documentos soltos, o calendário numa planilha e as métricas em cada rede social, o que dificulta manter a frequência de publicação e aprender com os resultados. O Kreato reúne banco de ideias, pipeline de produção em kanban, calendário editorial, anexos, métricas e trabalho em equipe num só lugar, com um modelo freemium (Gratuito, Creator e Studio).

Este relatório descreve primeiro a metodologia de trabalho e, em seguida, a arquitetura do SaaS como ela está no ar: componentes, onde rodam, o caminho de uma requisição, o fluxo de assinatura, o modelo de dados, a segurança, as extras e as camadas de nuvem. Depois vêm o quadro de entregas das atividades 7 a 11, uma ficha com evidências para cada atividade, as considerações finais e a declaração de autoria.

2 Metodologia

As atividades 7 e 8 foram feitas no repositório público do catálogo de filmes (`guilherme-dds/movie-catalog`), o mesmo do 1º bimestre. Na atividade 9 esse repositório foi copiado com todo o histórico (`git clone --bare` seguido de `git push --mirror`) para um repositório novo, `guilherme-dds/kreato`, onde o SaaS foi desenvolvido nas atividades 10 e 11. Como o histórico foi espelhado, os commits das atividades 7 e 8 têm o mesmo hash nos dois repositórios. A stack do Portainer, a VM e o domínio continuaram os mesmos; só o repositório de origem da stack foi trocado.

O desenvolvimento foi feito localmente com Docker Compose (Node.js 22 + TypeScript nos serviços, React 19 + Vite no frontend, Prisma como ORM), em commits pequenos e com mensagens no padrão *Conventional Commits*, e publicado no GitHub. O backlog do produto foi registrado como issues no repositório novo. O Stripe foi usado sempre em modo de teste, com o cartão `4242 4242 4242 4242`, e os webhooks foram testados localmente com a Stripe CLI (`stripe listen --forward-to`) e, em produção, pelo endpoint cadastrado no painel do Stripe.

As datas e horas de entrega foram extraídas do histórico de commits com `git log --date=format:"%d/%m/%Y %H:%M"` e conferidas na página de cada commit no GitHub. Da atividade 9 em diante, a data considerada é a da issue de entrega que menciona o professor, obtida com `gh issue list --json createdAt` (o GitHub guarda em UTC; os horários deste relatório estão convertidos para o horário de Brasília, UTC−3). Os prints mostram as páginas dos commits e das issues com data e hora visíveis, o README com a menção ao professor, o histórico de commits e o sistema em execução.

3 Arquitetura do SaaS

O sistema descrito aqui é o que está na stack: todos os componentes abaixo existem no `docker-compose.yml` do repositório `kreato`, exceto o MariaDB (servidor de banco da disciplina, acessado pelas variáveis `DATABASE_*`) e os serviços externos (Stripe e o provedor de e-mail).

3.1 Visão geral e componentes

O único ponto de entrada público é o Nginx, publicado na porta `8209` da VM (variável `KREATO_PORT`) e exposto pelo domínio da disciplina. Ele roteia por caminho para os serviços, que só escutam na rede interna do Docker (`expose`, sem `ports`). O Stripe chama o backend de fora para dentro apenas pelo webhook, que também passa pelo domínio e pelo Nginx.

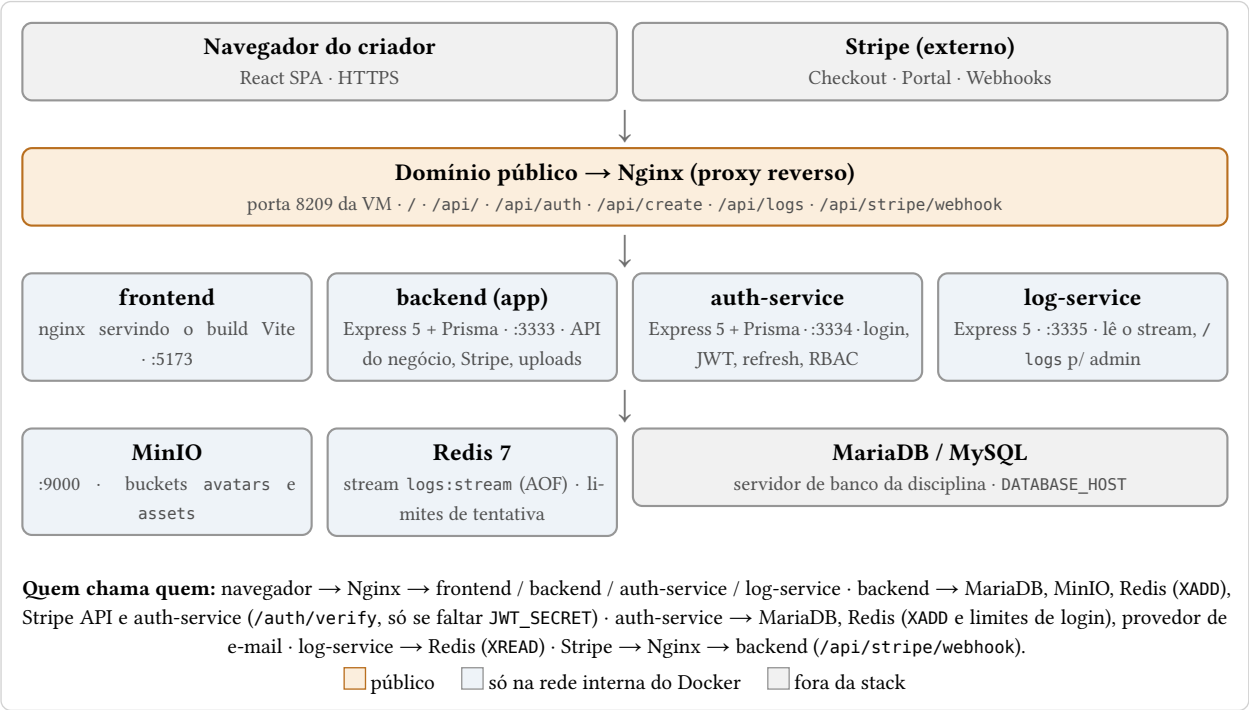


Figura 1: Diagrama de componentes e microsserviços do Kreato

Componente	Responsabilidade	Tecnologia / imagem	Público?
nginx	ponto único de entrada; roteia por caminho, cabeçalhos de segurança, gzip, upload até 110 MB	`nginx` (build próprio)	sim (domínio, porta 8209)
frontend	telas do produto e página pública de entrada (SPA)	React 19 + Vite, servido por nginx	via Nginx (`/`)
backend (app)	API do negócio: ideias, conteúdos, calendário, anexos, métricas, equipes, planos, Stripe, admin	Node 22 + Express 5 + Prisma 7	via Nginx (`/api/`)
auth-service	cadastro, login, JWT (15 min) + refresh token em cookie (7 dias), logout, redefinição de senha, papel (`role`)	Node 22 + Express 5 + Prisma 7 + bcryptjs	via Nginx (`/api/auth`, `/api/create`)

Componente	Responsabilidade	Tecnologia / imagem	Público?
log-service	consome os eventos de negócio e auditoria e expõe o histórico e as estatísticas ao admin	Node 22 + Express 5 + ioredis	via Nginx (`/api/logs`, só admin)
Redis	stream `logs:stream` com os eventos (`XADD`/`XREAD`) e contadores de rate limit	`redis:7.4-alpine`, AOF em volume	não (interno)
MariaDB	usuários, workspaces, ideias, conteúdos, métricas, planos, eventos do Stripe	MySQL/MariaDB (servidor da disciplina)	não
MinIO	fotos de perfil (`avatars`) e anexos dos conteúdos (`assets`)	MinIO (imagem fixada por digest)	não (arquivos servidos pelo backend)
Stripe (externo)	checkout de assinatura, portal do cliente, cobrança recorrente, webhooks	serviço SaaS (modo de teste)	webhook → Nginx → backend
E-mail (externo)	convites, redefinição de senha e aviso de cobrança recusada	Brevo (ou Mailtrap)	não (saída)

3.2 Onde roda

O Kreato roda na mesma stack do Portainer usada desde o 1º bimestre, na VM da disciplina, agora puxando o código do repositório novo `guilherme-dds/kreato` (branch `main`, arquivo `docker-compose.yml` na raiz). Na implantação, o Portainer faz o build das imagens de cada serviço a partir dos `Dockerfile` do repositório; o backend sincroniza o schema do banco com o Prisma no primeiro start (`docker-entrypoint.sh`) antes de abrir a porta.

- **Stack no Portainer:** TODO nome da stack. **URL pública:** https://SEU-SUBDOMINIO.
- **Containers:** `kreato-nginx`, `kreato-frontend`, `kreato-backend`, `kreato-auth-service`, `kreato-log-service`, `kreato-redis`, `kreato-minio`, todos com `restart: unless-stopped`.
- **Saúde e ordem de subida:** cada serviço tem *healthcheck* (`/health` nos serviços Node, `redis-cli ping`, `wget` no frontend) e o Nginx só sobe depois que frontend, backend, auth-service e log-service estão saudáveis.
- **Volumes:** `minio-data` (arquivos) e `redis-data` (stream de auditoria com AOF).
- **Rede:** rede padrão da stack com sub-rede fixa `10.209.0.0/24` (variável `KREATO_SUBNET`), para não esgotar os pools automáticos do Docker na VM compartilhada.
- **Limites de memória:** o `docker-compose.yml` não define `mem_limit`; TODO informe aqui os limites configurados na stack/VM, se houver.
- **Variáveis de ambiente (só os nomes; os valores ficam na stack do Portainer):** `DATABASE_URL`, `DATABASE_HOST`, `DATABASE_USER`, `DATABASE_PASSWORD`, `DATABASE_NAME`, `DATABASE_PORT`, `JWT_SECRET`, `COOKIE_SECURE`, `NODE_ENV`, `KREATO_PORT`, `KREATO_SUBNET`, `FRONTEND_URL`, `VITE_API_BASE_URL`, `TRUST_PROXY`, `CORS_ORIGINS`, `ENABLE_API_DOCS`, `EMAIL_PROVIDER`, `BREVO_API_KEY`, `BREVO_SENDER_EMAIL`, `MAILTRAP_TOKEN`, `MAILTRAP_SENDER_EMAIL`, `MINIO_ROOT_USER`,

`MINIO_ROOT_PASSWORD`, `MINIO_BUCKET_NAME`, `STRIPE_SECRET_KEY`, `STRIPE_PUBLISHABLE_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_CREATOR`, `STRIPE_PRICE_STUDIO`, `STRIPE_PRICE_ID`.

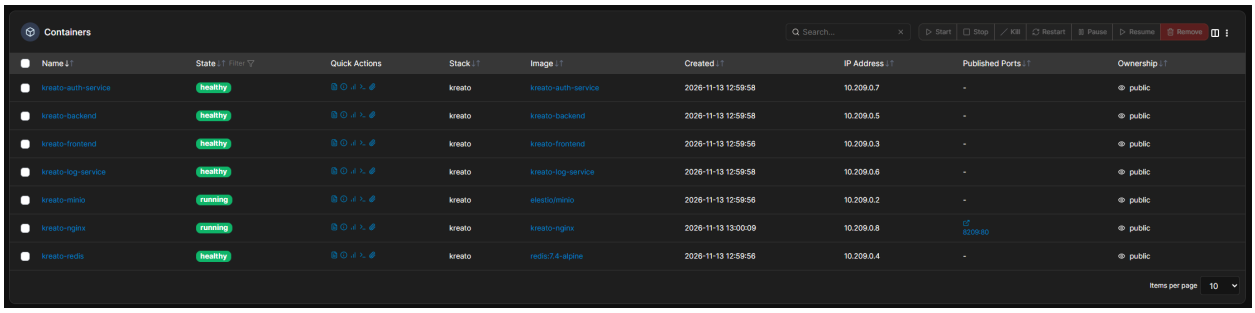


Figura 2: Stack do Kreato no Portainer com os containers em execução

3.3 Fluxo de uma requisição

Ação escolhida: **um roteirista move um conteúdo da coluna "Ideia" para "Roteiro" no pipeline** (`PATCH /api/conteudos/:id/etapa`).

1. **Navegador:** o card é arrastado no kanban; o frontend envia `PATCH /api/conteudos/42/etapa` com o cabeçalho `Authorization: Bearer <JWT>` e `X-Workspace-Id` do workspace selecionado.
2. **Domínio → Nginx:** a requisição chega ao Nginx, que acrescenta `X-Forwarded-For`/`X-Forwarded-Proto` e a encaminha, pelo prefixo `/api/`, para `backend:3333` na rede interna.
3. **Sessão (auth):** o `AuthMiddleware` valida o JWT emitido pelo auth-service. Como os dois serviços compartilham o `JWT_SECRET`, a verificação é local; sem o segredo, o backend pergunta ao auth-service (`POST /auth/verify`). Token vencido → `401`, e o frontend renova a sessão em `/api/auth/refresh` com o refresh token do cookie `httpOnly`.
4. **Workspace e papel:** o `WorkspaceMiddleware` confirma no banco que o usuário é membro do workspace pedido e lê o papel (`ROTEIRISTA`) e o plano do dono. Se não for membro: `403 WORKSPACE_FORBIDDEN`.
5. **Regra de negócio:** `canMoveContent` verifica se o papel pode levar o conteúdo de `IDEIA` para `ROTEIRO` (roteirista só circula entre Ideia e Roteiro; editor entre Gravação e Edição; dono/gestor em todas). Fora da faixa → `403`.
6. **MariaDB:** o Prisma atualiza `Conteudo.etapa` e a `posicao`, sempre com `where: { id, workspaceId }`, para que um id de outro workspace nunca seja encontrado.
7. **Log:** o backend publica o evento `CONTENT_STAGE_CHANGED` (com de/para, usuário, IP e horário) no Redis com `XADD logs:stream`; o log-service lê com `XREAD` e o evento aparece no painel admin. Negações geram `PERMISSION_DENIED`.
8. **Resposta:** `200` com o conteúdo atualizado; o kanban confirma a posição do card.

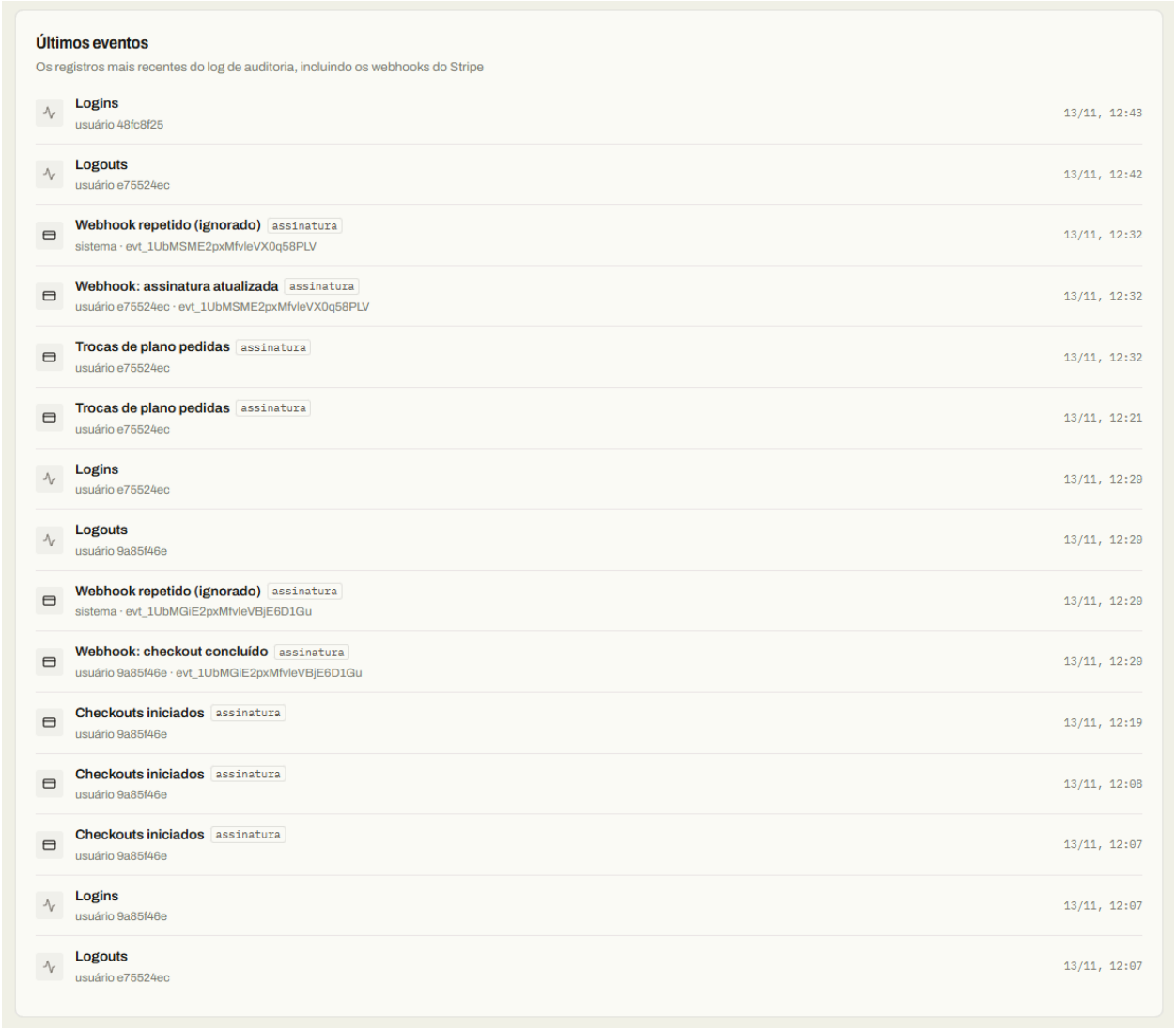


Figura 3: Evento do pipeline registrado no log-service (painel admin, Últimos eventos)

3.4 Fluxo de assinatura

O Stripe é a fonte da verdade da assinatura; o banco guarda uma cópia (`plano`, `statusAssinatura`, `valorMensalCentavos`, `stripeCustomerId`, `stripeSubscriptionId`) mantida **só pelos webhooks**. O retorno do navegador após o checkout não muda o plano: a tela apenas espera o webhook chegar.



Figura 4: Fluxo de assinatura: checkout → webhook → plano no banco → limites na API

Eventos tratados no webhook e o efeito de cada um:

Evento	O que muda no banco
`checkout.session.completed`	guarda o customer e a subscription no usuário e libera o plano pago
`customer.subscription.updated`	atualiza plano (pelo Price ID), status (`active`, `past_due`, `canceled`…) e valor mensal; cobre a troca de plano pelo portal
`customer.subscription.deleted`	volta o usuário para o plano Gratuito (cancelamento)
`invoice.payment_failed`	marca a assinatura como inadimplente, suspende o plano pago e avisa o usuário por e-mail e no app

- **Validação:** o endpoint recebe o corpo bruto e chama `stripe.webhooks.constructEvent` com o cabeçalho `Stripe-Signature` e o `STRIPE_WEBHOOK_SECRET`. Assinatura ausente ou inválida ⟶ `400`; sem segredo configurado o endpoint recusa tudo (`503`), para que ninguém ative um plano com um evento forjado.
- **Ordem dos eventos:** o estado da assinatura é sempre buscado na API do Stripe no momento do processamento, então a ordem de chegada não importa.
- **Evento repetido:** o `id` de cada evento é gravado na tabela `StripeEvento` **na mesma transação** da mudança no usuário. Se o mesmo evento chegar de novo, ele é encontrado e a resposta é `200` sem efeito (log `STRIPE_WEBHOOK_DUPLICATE`); se duas entregas chegarem ao mesmo tempo, a chave primária faz só a primeira gravar. Em erro inesperado a resposta é `500`, nada é gravado e o Stripe reenvia depois.
- **Cancelamento:** o botão "Gerenciar assinatura" abre o portal do cliente do Stripe (`POST /api/stripe/portal`); ao cancelar, o Stripe envia `customer.subscription.deleted` e o usuário volta ao Gratuito. A troca entre Creator e Studio pelo portal chega como `customer.subscription.updated`.

3.5 Modelo de dados do domínio

O banco é compartilhado pelo backend e pelo auth-service (cada um com o seu `schema.prisma`, mantidos iguais por um script de verificação). Todas as tabelas do negócio pertencem a um `Workspace`; é por ele que o isolamento é feito.

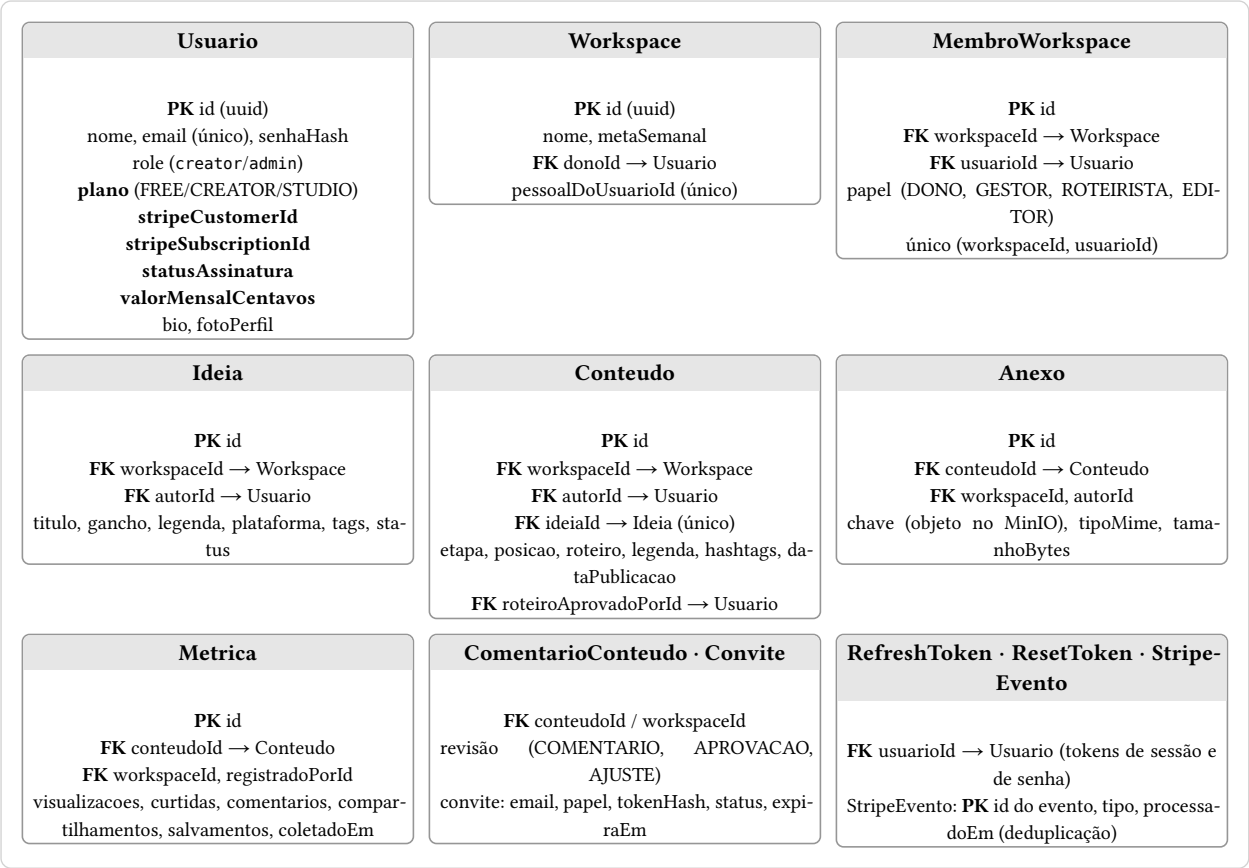


Figura 5: Modelo de dados do domínio (em negrito, as colunas que ligam usuário, plano e assinatura do Stripe)

O plano pertence ao **usuário** (`Usuario.plano`), e os limites de um workspace seguem o plano do **dono** (`Workspace.donoId`). A assinatura do Stripe se liga ao usuário por `stripeCustomerId` e `stripeSubscriptionId`; o webhook localiza o usuário pelo `client_reference_id` do checkout ou por esses ids. As exclusões usam `onDelete: Cascade` a partir de usuário e workspace, e `SetNull` nas autorias, para não perder conteúdo quando um membro sai.

3.6 Segurança

Há dois níveis de papel: a `role` da conta (`creator` para todo cadastro, `admin` atribuída direto no banco e lida do banco a cada acesso) e o papel do usuário dentro de cada workspace.

Ação	admin	Dono	Gestor	Roteirista	Editor
criar e editar ideias / roteiros (Ideia, Roteiro)	—	sim	sim	sim	não
aprovar roteiro e pedir ajustes	—	sim	sim	não	não
mover entre Gravação e Edição	—	sim	sim	não	sim
agendar e publicar	—	sim	sim	não	não
comentar, anexar arquivos, registrar métricas	—	sim	sim	sim	sim
convidar membros	—	sim	roteiristas e editores	não	não
assinar, trocar ou cancelar o plano	—	sim	não	não	não
ver dados de outro workspace	não	não	não	não	não

Ação	admin	Dono	Gestor	Roteirista	Editor
painel admin, métricas da plataforma e logs	sim	não	não	não	não

A `role` admin é da plataforma: ela dá acesso ao painel e aos logs, mas não dá acesso ao conteúdo dos workspaces dos clientes; dentro de um workspace o admin tem só o papel que tiver lá.

Isolamento por usuário. Toda rota do negócio passa por `AuthMiddleware` ⟶ `WorkspaceMiddleware`, que só aceita um workspace do qual o usuário é membro (senão `403` e evento `PERMISSION_DENIED`). Depois disso, todas as consultas do Prisma filtram por `workspaceId: req.workspaceId` junto com o id do registro (por exemplo, `where: { id, workspaceId }`), então tentar abrir a ideia de outro cliente pelo id retorna `404`, não o dado. Anexos não têm URL pública no MinIO: são lidos pelo backend em `/api/anexos/:id/arquivo` depois da mesma checagem. Os testes automatizados do backend cobrem permissões por papel, vagas por plano e cobrança.

Sessão e superfície da API. Senhas com bcrypt; JWT de 15 minutos e refresh token de 7 dias em cookie `httpOnly` (`Secure` atrás de HTTPS), trocado a cada renovação; limite de tentativas de login no Redis; CORS restrito ao próprio domínio; cabeçalhos de segurança no Nginx (`X-Frame-Options`, `nosniff`, `Referrer-Policy`, CSP no frontend); Swagger desligado em produção por padrão (`ENABLE_API_DOCS=false`).

Segredos. Nenhum valor real está no repositório: o `.env` está no `.gitignore` e o `.env.example` traz só os nomes das variáveis. Em produção, todos os segredos (`JWT_SECRET`, senhas do banco e do MinIO, chaves do Stripe e do e-mail) são cadastrados como variáveis da stack no Portainer. O compose recusa subir sem credenciais do MinIO (`${MINIO_ROOT_USER:?…}`) em vez de usar uma senha padrão.

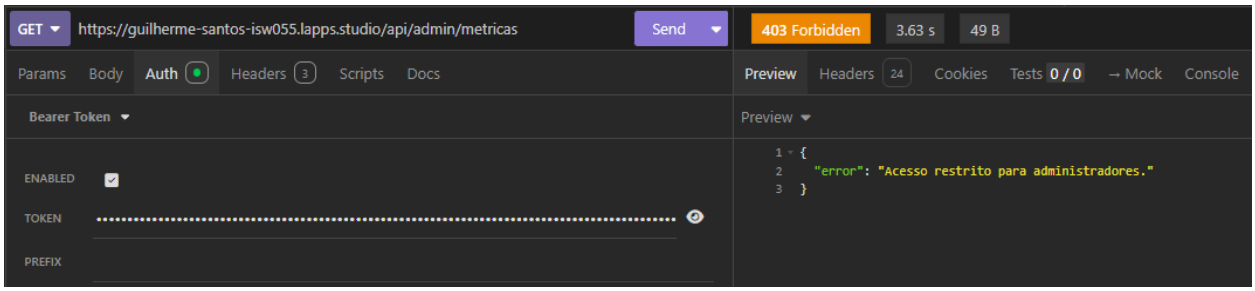


Figura 6: Usuário sem permissão recebendo 403 (e o evento `PERMISSION_DENIED` no log)

3.7 Observabilidade, CI/CD e documentação da API

- **Health checks:** os três serviços Node expõem `/health` e todos os containers têm *healthcheck* no compose, que também controla a ordem de subida da stack.
- **Observabilidade:** o painel administrativo mostra usuários por plano, assinaturas ativas, MRR (soma de `valorMensalCentavos` das assinaturas ativas), uso da plataforma e os últimos eventos do log-service, incluindo os webhooks do Stripe; o log-service oferece `GET /api/logs/stats?dias=30` com estatísticas de uso e segurança por dia.
- **CI/CD:** workflow do GitHub Actions (`.github/workflows/ci.yml`) a cada push na `main` e em pull requests: `npm ci`, geração do cliente Prisma, checagem de tipos (`tsc --noEmit`) de backend, auth-service e log-service, testes do backend (`npm test`), lint e build do frontend, e validação do `docker-compose.yml` (`docker compose config`). A implantação é feita pelo Portainer a partir do repositório.
- **Documentação da API:** Swagger/OpenAPI no backend (`/api/docs`), no auth-service (`/auth/apidocs`) e no log-service (`/log/apidocs`), habilitados por `ENABLE_API_DOCS`.

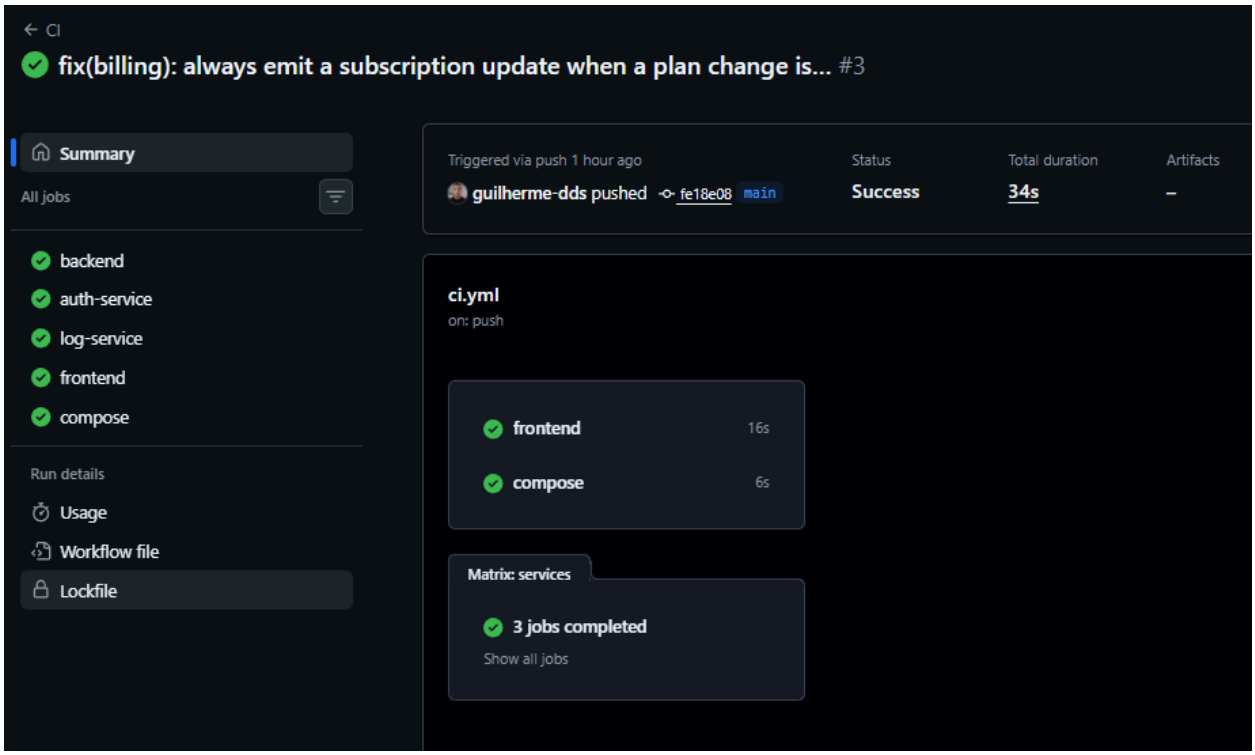


Figura 7: Execução do workflow de CI no GitHub Actions

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no Kreato	Quem gerencia
IaaS	a VM da disciplina com CPU, memória, disco e rede, o sistema operacional e o Docker Engine onde a stack roda; o domínio e o acesso de rede até a porta 8209	professor / Fatec (infraestrutura da disciplina)
PaaS	o Portainer (implantação da stack a partir do Git, variáveis, logs dos containers); o runtime de contêineres e os serviços de apoio usados como plataforma: MariaDB (servidor da disciplina), Redis e MinIO; o GitHub Actions como plataforma de CI	Portainer e servidor de banco: professor / Fatec; Redis, MinIO e a definição da stack: eu; GitHub Actions: GitHub
SaaS	o próprio Kreato, entregue pronto pelo navegador ao criador de conteúdo, com assinatura mensal; e os SaaS que o Kreato consome: Stripe (pagamentos), Brevo/Mailtrap (e-mail) e GitHub (código e issues)	Kreato: eu (código, dados e operação); Stripe, Brevo e GitHub: os fornecedores

4 Quadro de entregas

O quadro é gerado a partir das fichas da seção seguinte. As datas realizadas vêm dos commits (atividades 7 e 8) e das issues de entrega que mencionam o professor (atividades 9 a 11), no horário de Brasília.

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	09/10/2026 12:12	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	16/10/2026 15:55	entregue
9	Fork e plano de negócio	23/10/2026	24/10/2026 12:37	entregue com atraso
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	06/11/2026 18:10	entregue
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	13/11/2026 12:44	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	09/10/2026 12:12
Situação	entregue
Evidência	GitHub — issue #8 + print do checkout concluído
Link	https://github.com/guilherme-dds/movie-catalog/issues/8

O que foi feito. Ainda no catálogo de filmes, foi criado o `StripeController` no backend com a criação de uma Checkout Session em modo de teste e o endpoint de webhook que, ao receber `checkout.session.completed`, marca o usuário como premium no banco (campos novos no `schema.prisma` do backend e do auth-service). Para dar valor ao plano, foram adicionadas listas personalizadas de filmes (`CustomListController`) e recursos exclusivos, e o frontend ganhou o modal de assinatura (`PremiumModal`), o selo de premium (`PremiumBadge`) e a área do plano na página de perfil. As chaves do Stripe entraram como variáveis de ambiente do compose, sem valores no repositório. No mesmo dia vieram duas correções: a checagem de JWT expirado (`f4afd6b`) e o `.dockerignore` (`b98ce3b`).

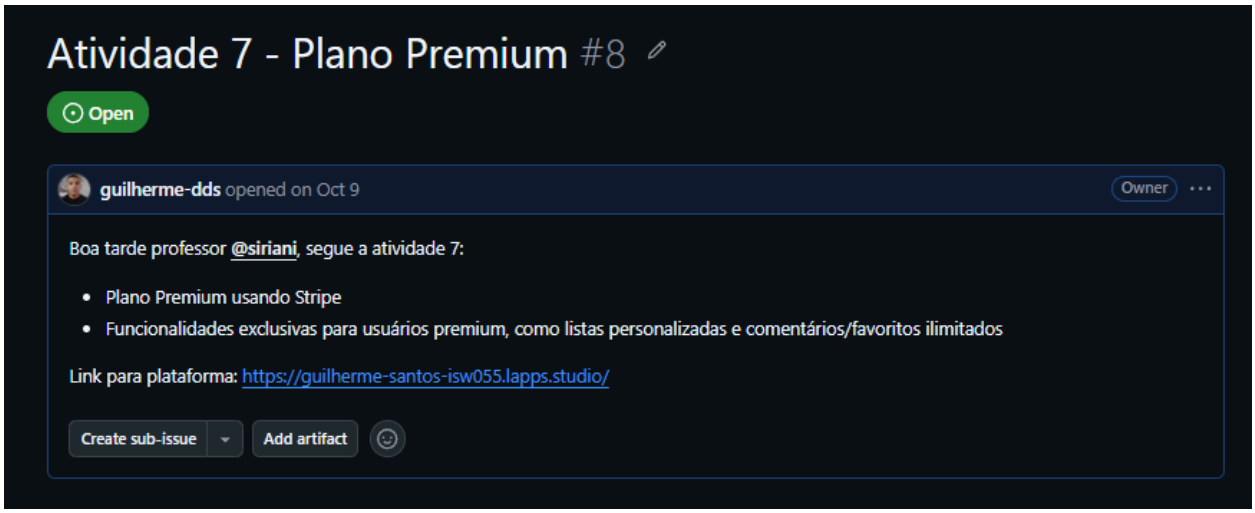


Figura 8: Atividade 7 — issue de entrega #8 mencionando o professor

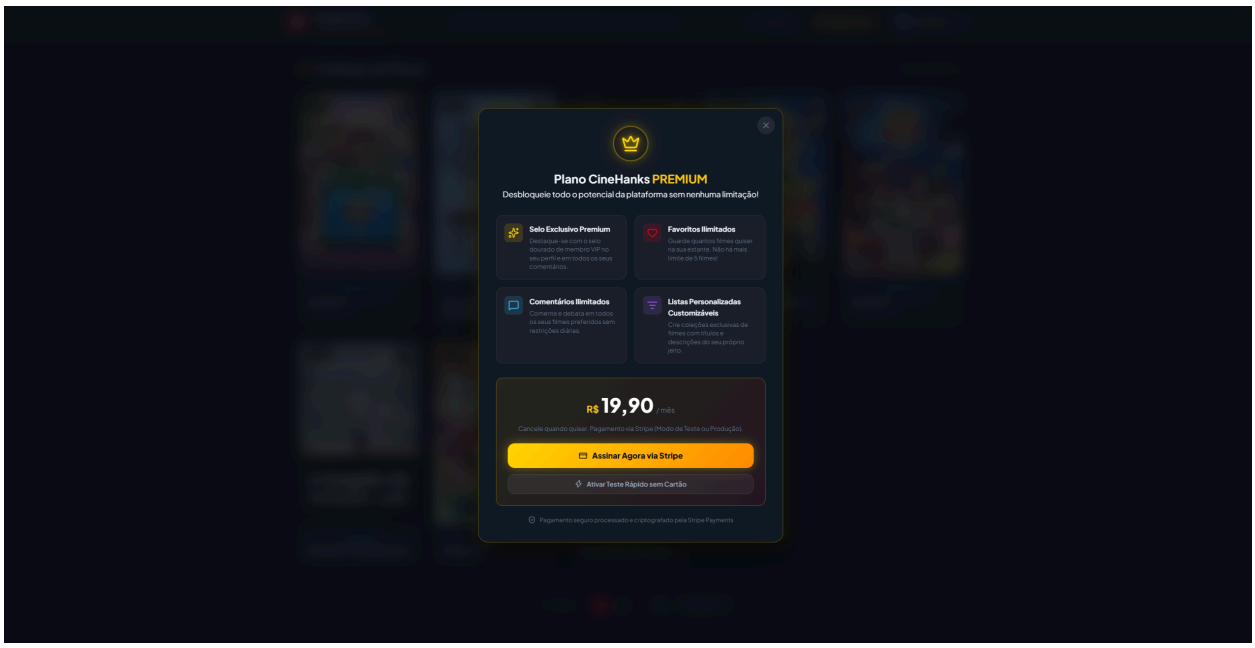


Figura 9: Atividade 7 — checkout da plataforma

Dificuldades e como foram resolvidas. TODO: revise com a sua experiência. Para testar o fluxo sem expor o servidor local, os eventos foram encaminhados com a Stripe CLI, e foi criado um modo simulado (`mockUpgrade`) para ativar o premium quando as chaves do Stripe não estão configuradas. Logo após a entrega apareceu um problema na checagem de tokens JWT expirados, corrigido no commit `f4afd6b` no mesmo dia.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	16/10/2026 15:55
Situação	entregue
Evidência	GitHub — issue #9 `documentacao.md` linkado no README

Link	https://github.com/guilherme-dds/movie-catalog/commit/c50d7324439787028079f4c40e7b25c909ecdfeb

O que foi feito. Primeiro foi adicionada ao README uma tabela que classifica cada peça do projeto nas camadas de nuvem: a VM como IaaS; Docker/Compose, Portainer e o banco como PaaS; Stripe e o provedor de e-mail como SaaS, cada uma com a justificativa (commit `260d498`). Em seguida foi escrita a documentação reversa em `documentacao.md` (commit `c50d732`, com o link no README): especificação do sistema do ponto de vista do usuário, diagrama de arquitetura em Mermaid com todos os serviços e portas, o mapeamento das regras de negócio no formato "se X, então Y" lidas do código e o modelo de dados. A ideia foi documentar o sistema **como ele estava**, não como deveria ser.

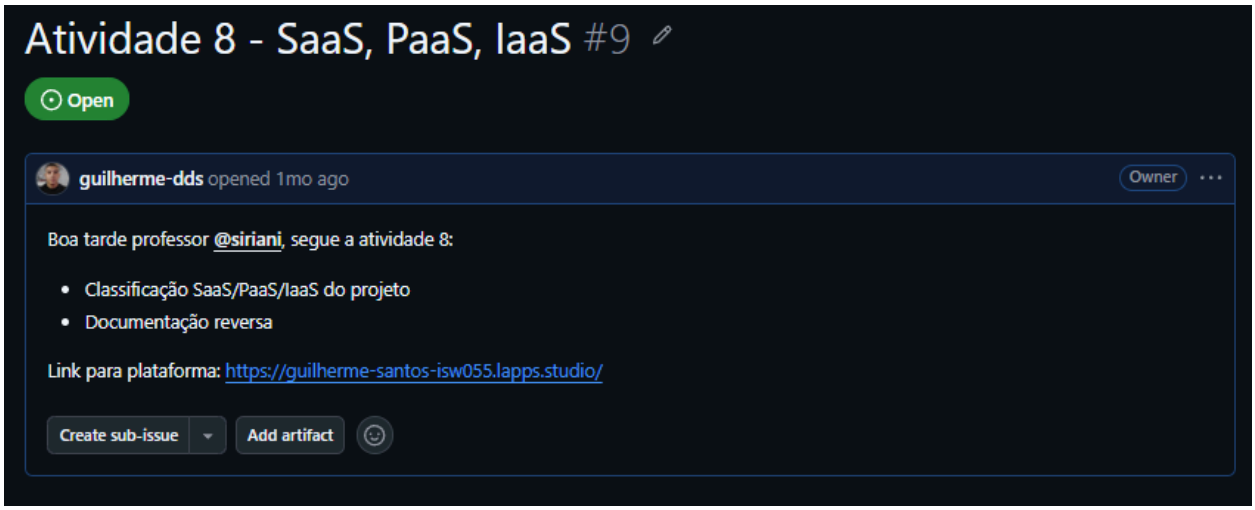
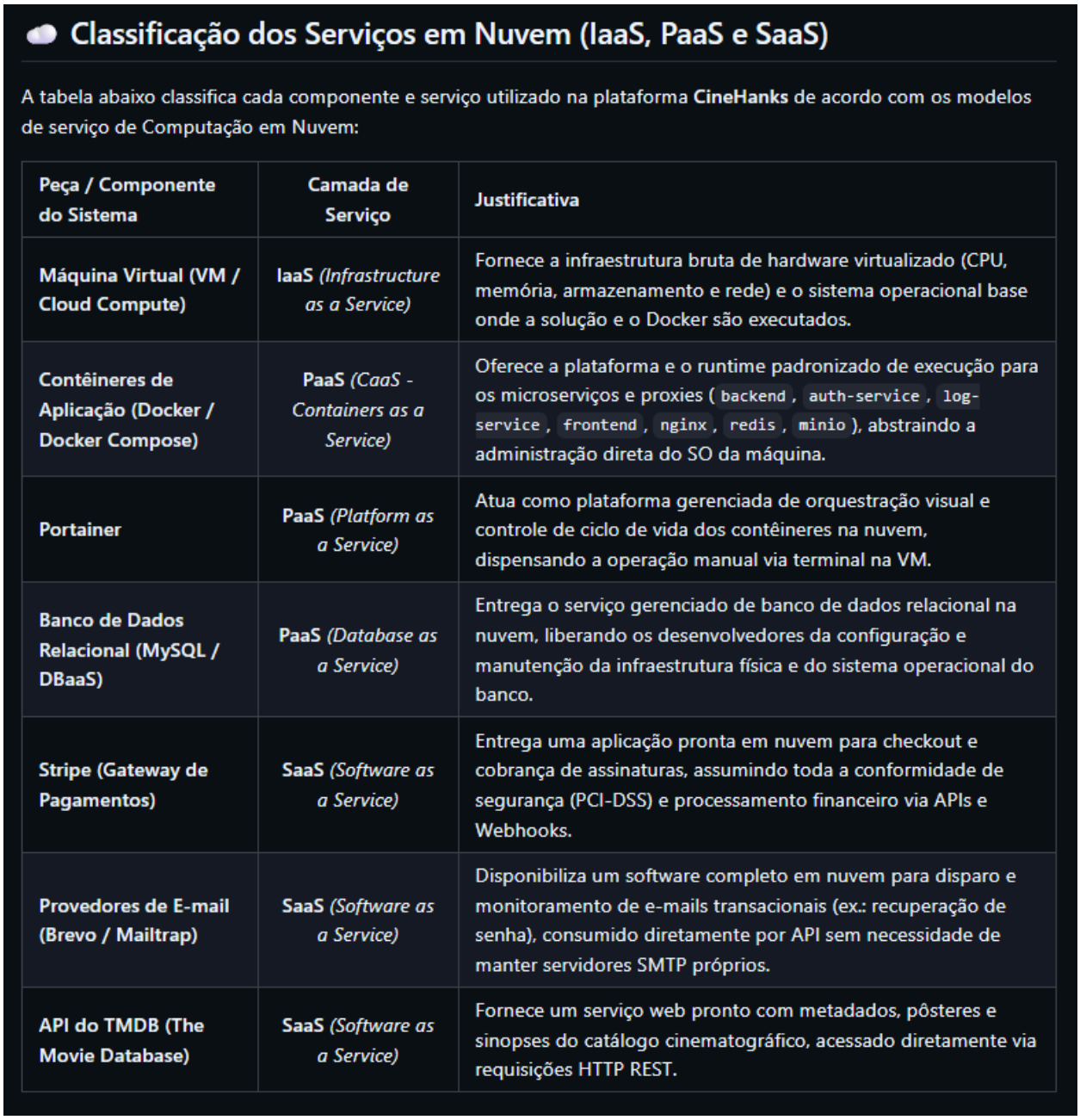


Figura 10: Atividade 8 — issue de entrega #9 mencionando o professor



Classificação dos Serviços em Nuvem (IaaS, PaaS e SaaS)

A tabela abaixo classifica cada componente e serviço utilizado na plataforma **CineHanks** de acordo com os modelos de serviço de Computação em Nuvem:

Peça / Componente do Sistema	Camada de Serviço	Justificativa
Máquina Virtual (VM / Cloud Compute)	**IaaS** *(Infrastructure as a Service)*	Fornece a infraestrutura bruta de hardware virtualizado (CPU, memória, armazenamento e rede) e o sistema operacional base onde a solução e o Docker são executados.
Contêineres de Aplicação (Docker / Docker Compose)	**PaaS** *(CaaS - Containers as a Service)*	Oferece a plataforma e o runtime padronizado de execução para os microserviços e proxies (`backend` , `auth-service` , `log-service` , `frontend` , `nginx` , `redis` , `minio`), abstraindo a administração direta do SO da máquina.
Portainer	**PaaS** *(Platform as a Service)*	Atua como plataforma gerenciada de orquestração visual e controle de ciclo de vida dos contêineres na nuvem, dispensando a operação manual via terminal na VM.
Banco de Dados Relacional (MySQL / DBaaS)	**PaaS** *(Database as a Service)*	Entrega o serviço gerenciado de banco de dados relacional na nuvem, liberando os desenvolvedores da configuração e manutenção da infraestrutura física e do sistema operacional do banco.
Stripe (Gateway de Pagamentos)	**SaaS** *(Software as a Service)*	Entrega uma aplicação pronta em nuvem para checkout e cobrança de assinaturas, assumindo toda a conformidade de segurança (PCI-DSS) e processamento financeiro via APIs e Webhooks.
Provedores de E-mail (Brevo / Mailtrap)	**SaaS** *(Software as a Service)*	Disponibiliza um software completo em nuvem para disparo e monitoramento de e-mails transacionais (ex.: recuperação de senha), consumido diretamente por API sem necessidade de manter servidores SMTP próprios.
API do TMDB (The Movie Database)	**SaaS** *(Software as a Service)*	Fornece um serviço web pronto com metadados, pôsteres e sinopses do catálogo cinematográfico, acessado diretamente via requisições HTTP REST.

Figura 11: Atividade 8 — README com a tabela IaaS/PaaS/SaaS e o link para a documentação

Dificuldades e como foram resolvidas. TODO: revise com a sua experiência. Várias regras de negócio só existiam implícitas no código (moderação de comentários, favoritos, expiração de tokens), e foi preciso ler os controllers e testar as rotas para descrevê-las corretamente. Classificar o Portainer e o banco também gerou dúvida entre IaaS e PaaS; a decisão foi pelo critério de quem administra o sistema operacional e o runtime.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026

Data realizada	24/10/2026 12:37
Situação	entregue com atraso
Evidência	GitHub — issue de entrega #6 (@siriani) + repositório novo + README do produto (commit `ff4e942`) + backlog nas issues #1 a #5
Link	https://github.com/guilherme-dds/kreato/issues/6

O que foi feito. O repositório `movie-catalog` foi espelhado com todo o histórico (`git clone --bare` + `git push --mirror`) no repositório novo `guilherme-dds/kreato`, criado em 23/10/2026 às 18:04. O README foi reescrito para o produto (commit `ff4e942`, 24/10 12:21): proposta de valor, Lean Canvas completo (problema, segmentos, proposta única de valor, solução, canais, receitas, custos, métricas-chave e vantagem injusta), modelo freemium com os planos Gratuito, Creator e Studio e o mapa de reaproveitamento de cada peça do projeto (login + RBAC ⟶ papéis do negócio, logs ⟶ eventos do ciclo de produção, MinIO ⟶ mídias dos conteúdos, Stripe ⟶ assinatura). O backlog do produto foi aberto como issues (#1 a #5): remover o catálogo e renomear o projeto, central de ideias e roteiros, pipeline e calendário, métricas e planos com Stripe.

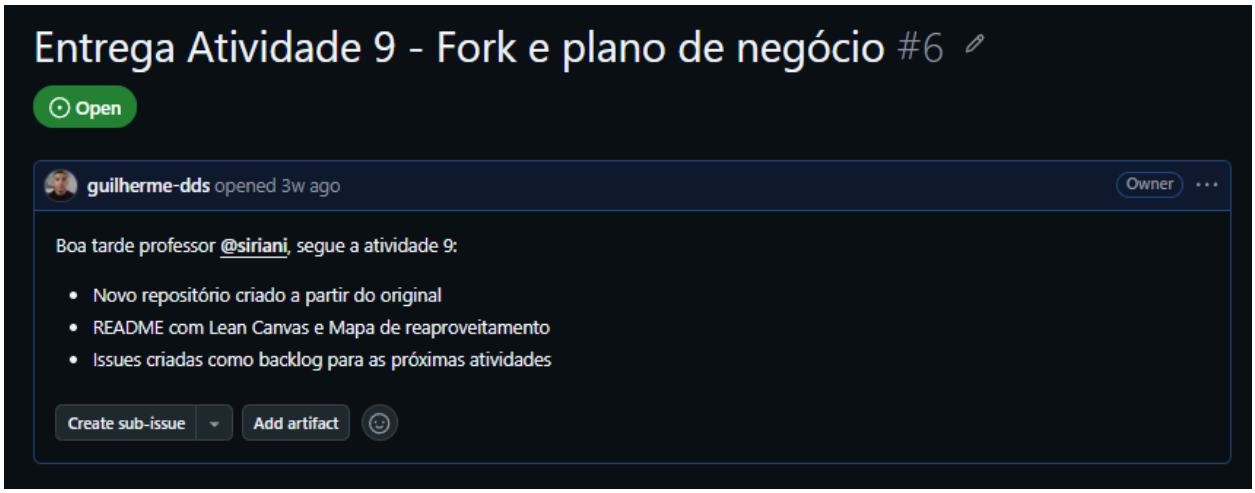


Figura 12: Atividade 9 — issue de entrega #6 mencionando o professor

🎯 Lean Canvas

Bloco	Descrição
1. Problema	1. Ideias, roteiros e calendários ficam espalhados em diferentes ferramentas. 2. Dificuldade para manter uma frequência consistente de publicações. 3. Falta de organização e acompanhamento do processo de produção e dos resultados dos conteúdos. **Alternativas atuais:** Notion, Trello, planilhas e anotações.
2. Segmentos de clientes	Criadores de conteúdo em geral, desde iniciantes até profissionais que publicam em plataformas como TikTok, Instagram, YouTube e outras redes sociais. Inclui influenciadores, produtores de conteúdo digital e pequenas equipes de criação.
3. Proposta única de valor	Tudo o que você precisa para planejar, organizar e acompanhar seus conteúdos em um só lugar. O Kreato centraliza o fluxo de produção para ajudar criadores a economizar tempo, manter a consistência e melhorar seus resultados.
4. Solução	1. Central de ideias e roteiros: registre temas, ganchos, legendas e organize suas ideias. 2. Gestão da produção: acompanhe cada conteúdo em um pipeline e planeje publicações com um calendário editorial. 3. Acompanhamento de resultados: registre métricas e compare o desempenho dos conteúdos para orientar futuras produções.
5. Canais	TikTok, Instagram e YouTube para divulgar o produto; parcerias com criadores de conteúdo; comunidades de criadores nas redes sociais; vídeos demonstrativos da plataforma; indicações entre usuários.
6. Fontes de receita	Modelo freemium com assinatura mensal. • **Gratuito:** funcionalidades básicas com limites de uso. • **Kreato Creator:** R$ 29–39/mês, com mais projetos e recursos de organização. • **Kreato Studio:** R$ 79–129/mês, com colaboração, permissões e ferramentas para equipes. *Valores iniciais a validar com potenciais clientes.*
7. Estrutura de custos	Hospedagem do frontend e backend; banco de dados; domínio e serviços de e-mail; taxas do processador de pagamentos; custos de APIs de IA, caso sejam utilizadas; divulgação, suporte, desenvolvimento e manutenção.
8. Métricas-chave	Número de usuários cadastrados; usuários ativos semanais; quantidade de conteúdos planejados e publicados; percentual de usuários que concluem o fluxo de produção; conversão de usuários gratuitos para pagos; receita recorrente mensal (MRR); retenção e cancelamento de assinaturas.
9. Vantagem injusta	Conhecimento sobre os desafios da produção de conteúdo e foco em centralizar o fluxo de trabalho do criador. Com o tempo, a vantagem poderá ser construída por meio de modelos personalizáveis, histórico de desempenho, recomendações adaptadas à rotina dos usuários e uma comunidade de criadores.

Figura 13: Atividade 9 — README do produto com o Lean Canvas e o backlog nas issues

Dificuldades e como foram resolvidas. O repositório novo foi criado dentro do prazo (23/10), mas o README do produto e a issue de entrega só foram concluídos na manhã seguinte, 24/10, o que caracteriza entrega com atraso. TODO: explique em uma frase o motivo do atraso. A parte mais trabalhosa foi definir o produto e o modelo de receita de forma que todas as peças já construídas tivessem um papel real no negócio, em vez de começar do zero.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	06/11/2026 18:10
Situação	entregue
Evidência	GitHub — issue de entrega #7 (@siriani) + commit `9fefd61` + URL no ar + prints da página pública, do 403 e dos eventos no log
Link	https://github.com/guilherme-dds/kreato/issues/7

O que foi feito. O projeto foi renomeado (`1523af5`) e o domínio de filmes removido (`961a087`). No lugar entraram as tabelas e a API do negócio: workspaces pessoais e de equipe, banco de ideias, pipeline de conteúdos em seis etapas (Ideia → Roteiro → Gravação → Edição → Agendado → Publicado) com kanban e editor de roteiro, calendário editorial, anexos no MinIO (bucket `assets`), métricas por conteúdo com página de análise e o dashboard do criador. Os papéis do negócio (Dono, Gestor, Roteirista, Editor) passaram a valer na API com permissões por etapa, convites por e-mail e comentários de revisão; toda consulta filtra pelo workspace do usuário e cada ação relevante gera um evento de negócio no Redis Stream (`IDEA_CREATED`, `CONTENT_STAGE_CHANGED`, `SCRIPT_APPROVED`, `METRIC_RECORDED`, `MEMBER_INVITED`…), além de `PERMISSION_DENIED` nas negações. A stack do Portainer passou a puxar o repositório novo; o último ajuste do dia (`9fefd61`) fixou a sub-rede da stack, porque a VM tinha esgotado os pools de endereços do Docker.

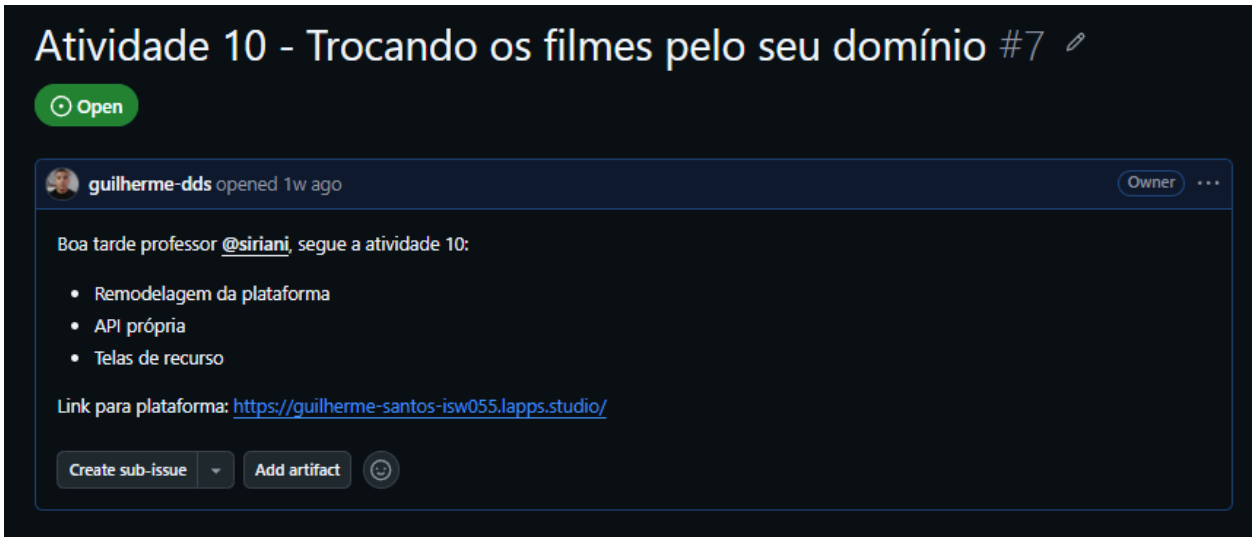


Figura 14: Atividade 10 — issue de entrega #7 mencionando o professor



Figura 15: Atividade 10 — página pública do Kreato (entrada com login e cadastro) no ar, na URL pública

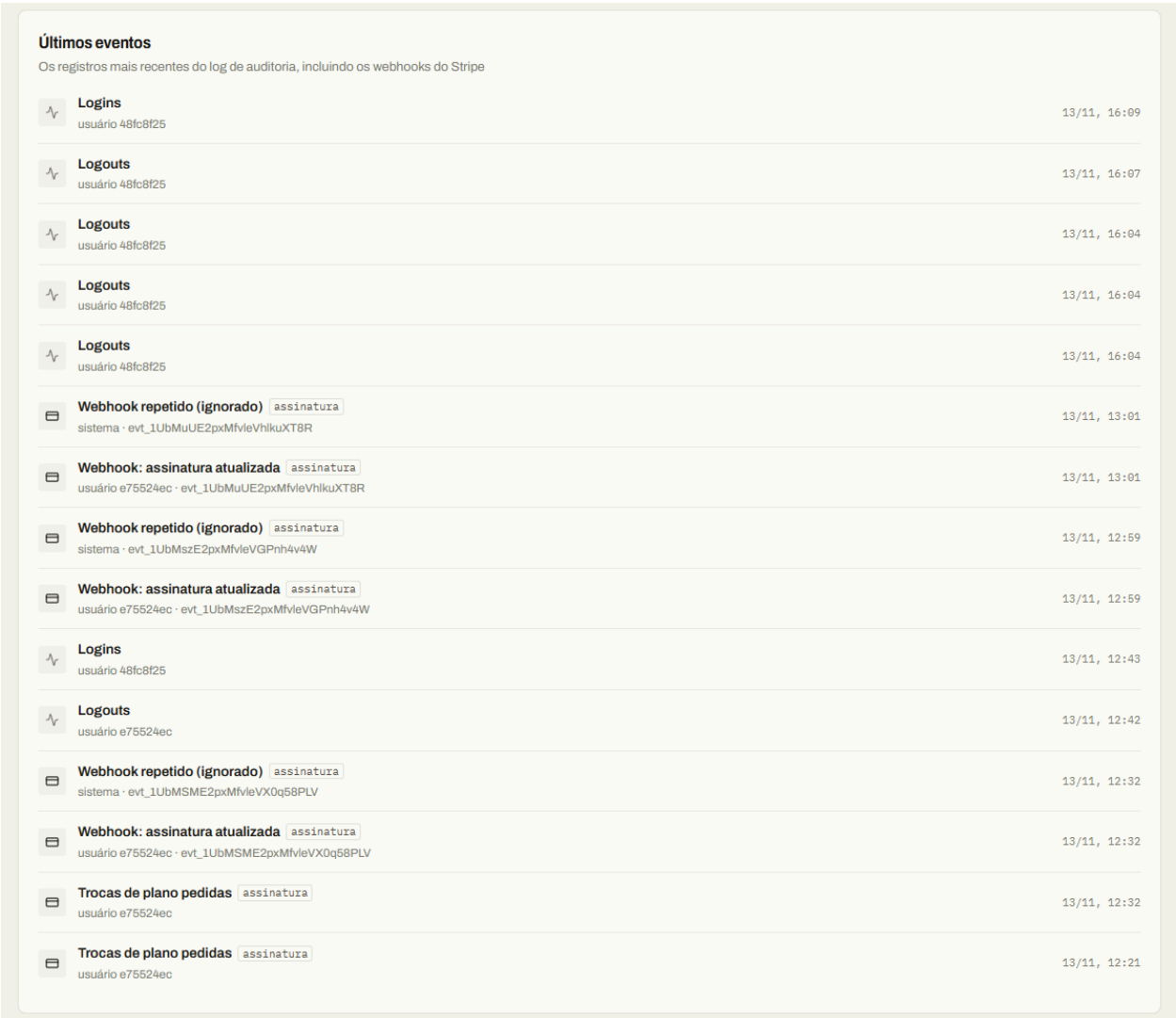


Figura 16: Atividade 10 — eventos de negócio no log (painel admin / log-service)

Dificuldades e como foram resolvidas. A maior foi a implantação: ao subir a stack nova a VM recusou criar a rede com o erro *all predefined address pools have been fully subnetted*, resolvido fixando uma sub-

-rede própria no compose (configurável por `KREATO_SUBNET`). No código, o desafio foi garantir o isolamento em todas as rotas; a solução foi centralizar a resolução do workspace num middleware e sempre consultar por `id` + `workspaceId`.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	13/11/2026 12:44
Situação	entregue
Evidência	GitHub — issue de entrega #8 (@siriani) + commits `aa7c9f3` e `fe18e08` + prints dos planos via Stripe e do painel de admin com informações dos planos
Link	https://github.com/guilherme-dds/kreato/issues/8

O que foi feito. O plano premium único virou três planos: Gratuito (R$ 0, 50 ideias, 15 conteúdos em produção, 100 MB, 30 dias de histórico, 1 membro), Kreato Creator (R$ 29,90/mês, ilimitado, 2 GB) e Kreato Studio (R$ 79,90/mês, 10 GB e até 5 membros). O checkout passou a criar assinaturas recorrentes (`mode=subscription`) com os Price IDs de cada plano, e o plano do usuário passou a ser alterado **somente** pelos webhooks (`checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted`, `invoice.payment_failed`), com validação da assinatura, deduplicação pela tabela `StripeEvento` e aviso de cobrança recusada. Foram adicionados o portal do cliente para trocar de plano e cancelar, os limites aplicados na API a cada requisição (`403 PLAN_LIMIT_REACHED` com a mensagem de como liberar), a página de planos com o uso atual e o painel admin com usuários por plano, assinaturas ativas, MRR real e os últimos eventos, inclusive os webhooks. Junto vieram endurecimento de segurança, paginação, health checks, testes e CI.

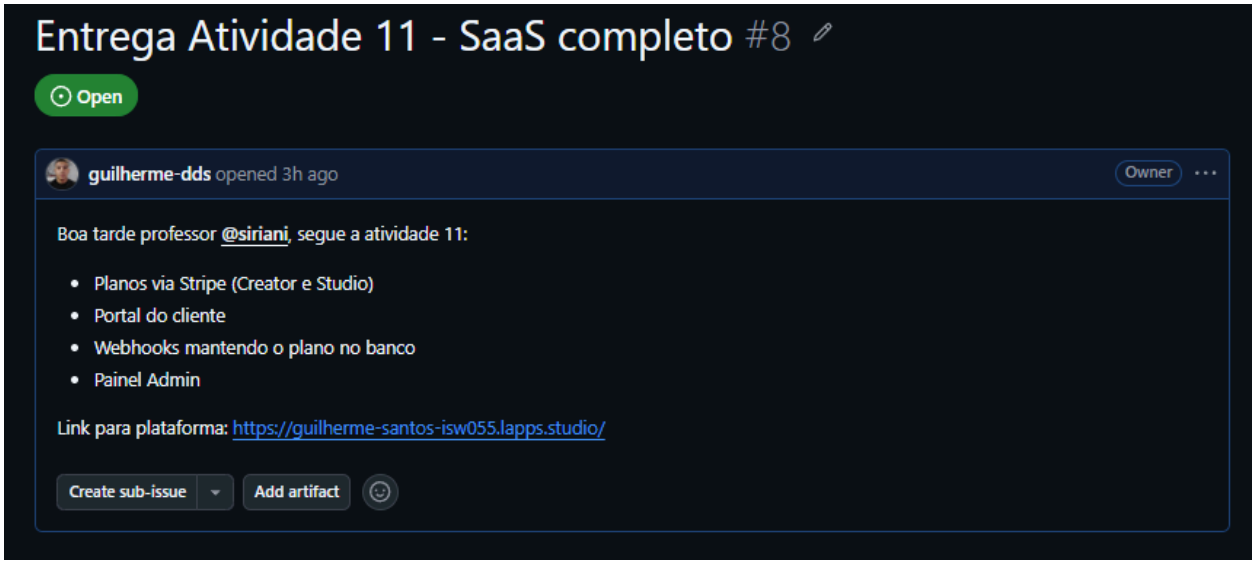


Figura 17: Atividade 11 — issue de entrega #8 mencionando o professor

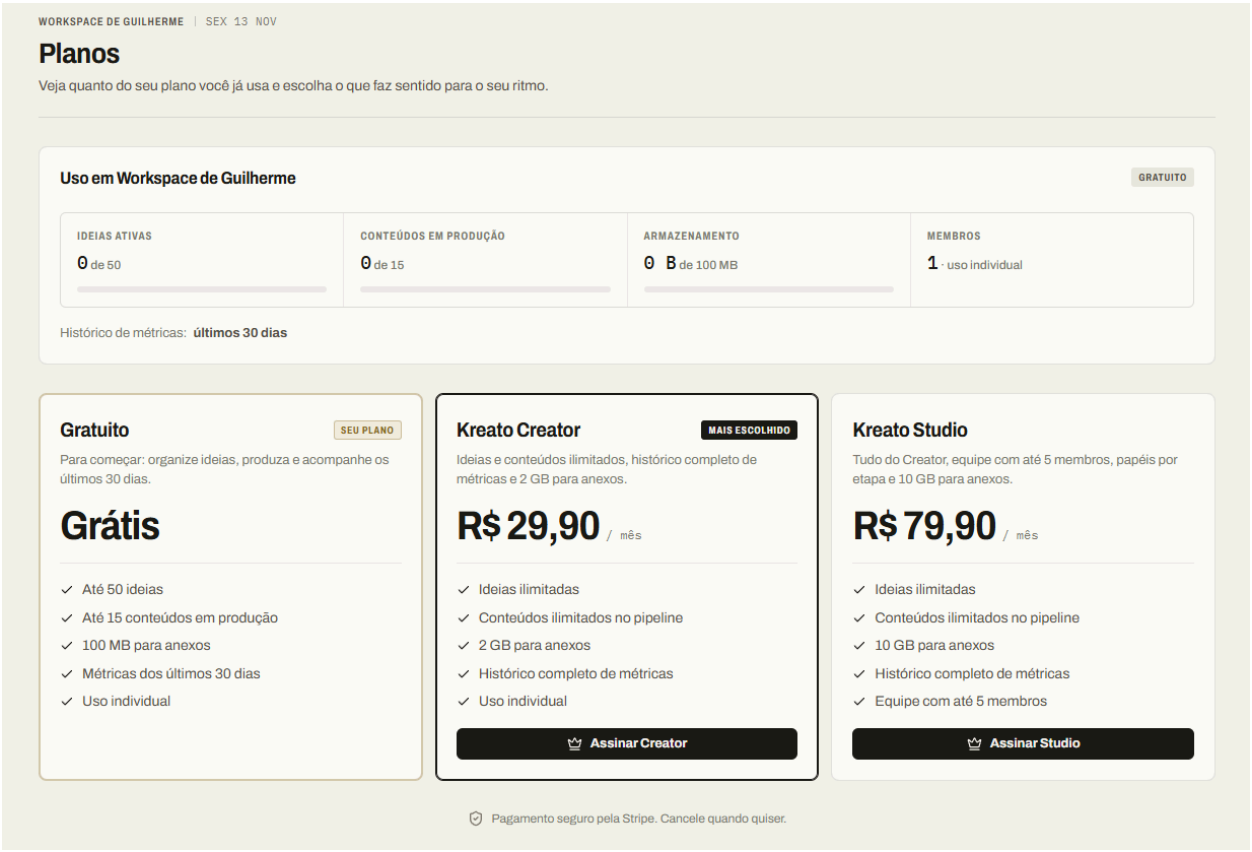


Figura 18: Atividade 11 — planos disponíveis com Stripe



Figura 19: Atividade 11 — painel admin com usuários por plano, MRR e eventos do Stripe

Dificuldades e como foram resolvidas. A primeira versão atualizava o plano quando o navegador voltava do checkout, o que deixava o banco divergente do Stripe em cancelamentos e falhas de pagamento. A solução foi tornar os webhooks a única fonte de mudança de plano e buscar o estado atual da assinatura no Stripe a cada evento, o que também resolveu eventos fora de ordem. Os reenvios do Stripe foram tratados gravando o id do evento na mesma transação da mudança. No dia da entrega, um último ajuste (`fe18e08`) garantiu que a troca de plano pelo portal sempre gere uma atualização da assinatura.

6 Considerações finais

O bimestre mostrou na prática a diferença entre ter funcionalidades e ter um produto. As peças construídas no 1º bimestre — autenticação separada, RBAC, auditoria com Redis, upload no MinIO — eram exercícios isolados; ao transformá-las no Kreato, cada uma precisou responder a uma pergunta de negócio: quem pode fazer o quê dentro de uma equipe, que evento importa registrar, quanto espaço cada plano libera. Também ficou claro o papel de cada camada de nuvem: a VM e a rede vêm da instituição, a plataforma (Portainer, Docker, banco) é compartilhada, e o que eu de fato opero é o SaaS e a sua configuração.

A parte mais difícil foi a cobrança. Integrar um checkout é simples; manter o banco coerente com o Stripe diante de cancelamentos, trocas de plano, pagamentos recusados, eventos repetidos e fora de ordem exigiu repensar o fluxo e tratar o webhook como a única fonte da verdade. A implantação numa VM compartilhada também trouxe problemas que não aparecem localmente, como o esgotamento das sub-redes do Docker.

Para o Kreato ser um produto de verdade ainda faltam: sair do modo de teste do Stripe (conta ativada, emissão de nota fiscal e termos de uso e privacidade adequados à LGPD), coletar métricas automaticamente pelas APIs do TikTok, Instagram e YouTube em vez do registro manual, backups automáticos do banco e do MinIO, monitoramento com alertas, HTTPS e domínio próprios, e validar os preços e limites dos planos com criadores reais, como o próprio Lean Canvas indica.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Guilherme Dias dos Santos

Pompeia, 13/11/2026